

[页面](#) / ... / [技术相关](#)

ThreadPoolExecutor 对线程池维护

由 朱明 创建, 最后修改于八月 31, 2022  55 views since 30 Aug 2022

-
- [一、如何让线程池自动关闭](#)
 - [如何关闭](#)
 - [具体代码逻辑](#)
- [二、关闭线程池测试](#)
- [三、如何选择拒绝策略](#)

一、如何让线程池自动关闭

如何关闭

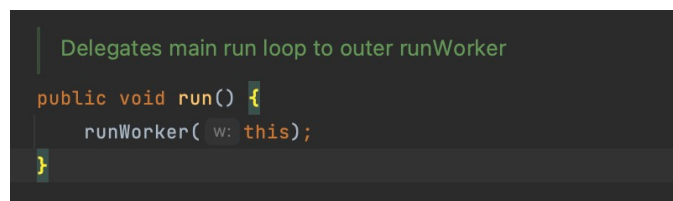
ThreadPoolExecutor对于核心线程，会为每个**核心**线程分配一个**监控线程**，在后台不断监控，该监控线程是线程池创建后无法关闭的**罪魁祸首**。

例如使用 `Executors.newFixedThreadPool(10)` 创建线程后，`stack`中始终会有10个核心线程，该10个核心线程分别有各自的维护逻辑。通过设置下面两个参数可以实现线程池自动关闭

```
executor.setKeepAliveTime(10, TimeUnit.SECONDS);
executor.allowCoreThreadTimeOut(true);
```

具体代码逻辑

其中`java.util.concurrent.ThreadPoolExecutor` 中的`run`方法变是用来**维护核心线程**的入口。



```
Delegates main run loop to outer runWorker

public void run() {
    runWorker( w: this);
}
```

下面主要介绍下`runWorker`函数，也就是**维护核心线程的主要流程**。

整个方法的逻辑，就是当前Worker不断死循环获取核心线程的队列里面是否有任务，有，就加锁然后执行任务。无，就阻塞等待获取任务。如果要跳出整个死循环，执行`processWorkerExit`的唯一入口应该在第10行的`getTask()`中。

```
1
2  final void runWorker(Worker w) {
3      Thread wt = Thread.currentThread();
4      // 获取目前运行中核心线程任务队列，并获取头节点
5      Runnable task = w.firstTask;
6      w.firstTask = null;
7      w.unlock(); // allow interrupts
8      boolean completedAbruptly = true;
9      try {
10         // 死循环，主要是维护核心线程
11         while (task != null || (task = getTask()) != null) {
12             w.lock();
13             // If pool is stopping, ensure thread is interrupted;
14             // if not, ensure thread is not interrupted. This
15             // requires a recheck in second case to deal with
16             // shutdownNow race while clearing interrupt
17             if ((runStateAtLeast(ctl.get(), STOP) ||
18                 (Thread.interrupted() &&
19                  runStateAtLeast(ctl.get(), STOP))) &&
20                 !wt.isInterrupted())
21                 wt.interrupt();
22             try {
23                 beforeExecute(wt, task);
24                 Throwable thrown = null;
25                 try {
26                     task.run();
27                 } catch (RuntimeException x) {
28                     thrown = x; throw x;
29                 } catch (Error x) {
30                     thrown = x; throw x;
31                 } catch (Throwable x) {
32                     thrown = x; throw new Error(x);
33                 } finally {
34                     afterExecute(task, thrown);
35                 }
36             } finally {
37                 task = null;
38                 w.completedTasks++;
39                 w.unlock();
40             }
41         }
42         completedAbruptly = false;
```

```
43         } finally {
44             processWorkerExit(w, completedAbruptly);
45         }
    }
```

接下来主要了解getTask的流程，找出如何退出无限循环的入口。

第12行是一个退出条件，但需要手动去关闭线程池。

第25行是一个退出条件，主要是关闭掉超出核心线程数的线程，由于一般初始化**核心线程数**和**最大线程数**一样，不存在这种情况。

第36行是一个退出条件，接下来主要介绍这条语句

Runnable r = timed ? workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS) : workQueue.take();

timed赋值语句**boolean timed = allowCoreThreadTimeOut || wc > corePoolSize;** 其中因为初始化**核心线程数**和**最大线程数**一样所以始终为**false**，即**timed = allowCoreThreadTimeOut**。

poll(keepAliveTime, TimeUnit.NANOSECONDS)的动作含义是阻塞等待一段时间，然后返回**workQueue**的头节点。

而对于**workQueue.take()**如果队列中没有数据，则线程会永久**wait**，等待数据。

至此可以推测在没有其它任务进入的情况下，该**核心线程**可能在这一步一直等待**workQueue**数据，这样核心线程永远释放不了。也就出现了一些事实任务的stack中充满着等待条件旧线程的原因。

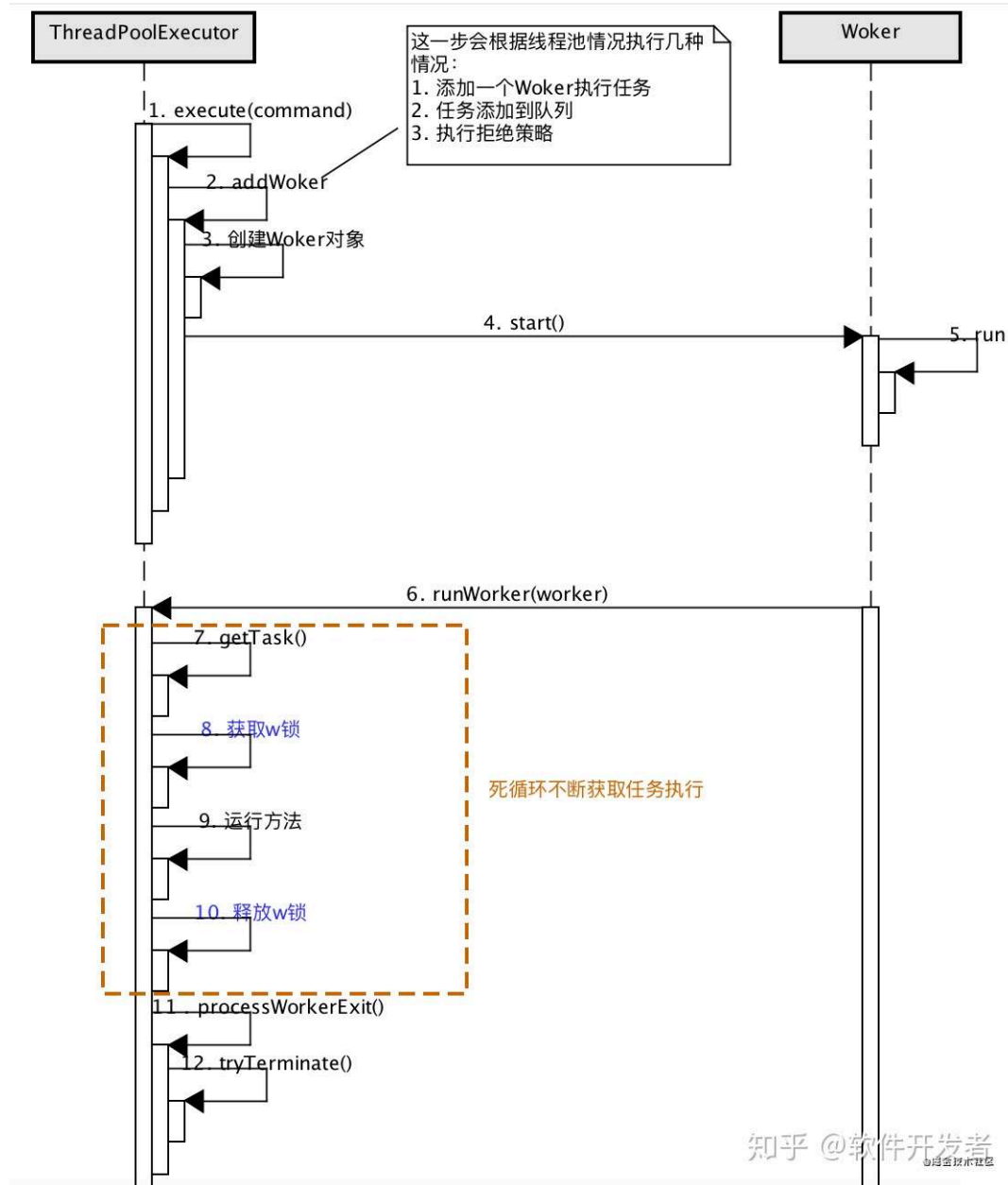
如果**timed=true并且队列没有新线程**，那么变会退出循环返回null，runWorker的死循环 while (task != null || (task = getTask()) != null)的条件被打断，维护该核心线程的任务变停止，该核心线程退出，等所以核心线程退出后，线程池也就自动清空。

```
1
2 private Runnable getTask() {
3     boolean timedOut = false; // Did the last poll() time out?
4     //死循环
5     for (;;) {
6         int c = ctl.get();
7         int rs = runStateOf(c);
8
9         // Check if queue empty only if necessary.
10        // 线程池关闭，或者已经停止并且队列为空
11        if (rs >= SHUTDOWN && (rs >= STOP || workQueue.isEmpty())) {
12            decrementWorkerCount();
13            return null;
14        }
15
16        int wc = workerCountOf(c);
17
18        // Are workers subject to culling?
19        // 超时时间的标识，是否设置了核心线程数的超时时间 或者 当前线程数量是否大于核心线程数
20        boolean timed = allowCoreThreadTimeOut || wc > corePoolSize;
21
22        //如果线程池运行的线程数量如果大于核心线程数，多出来的那部分线程是需要被回收的。
23        if ((wc > maximumPoolSize || (timed && timedOut))
24            && (wc > 1 || workQueue.isEmpty())) {
25            if (compareAndDecrementWorkerCount(c))
26                return null;
27            continue;
28        }
29
30        try {
31            // 如果timed为false，则一直阻塞等待，直到获取到元素，然后返回
32            // 如果timed为true，则一直阻塞等待keepAliveTime超时后返回，
33            Runnable r = timed ?
34                workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS) :
35                workQueue.take();
36            if (r != null)
37                return r;
38            timedOut = true;
39        } catch (InterruptedException retry) {
40            timedOut = false;
41        }
42    }
    }
```

下图为线程池的生命周期

其中前5个步骤对应着Executors有关内存泄漏分析中提到的创建线程流程。

后面的步骤是线程池对核心线程的维护逻辑，对应着上面的代码逻辑。



参考：

<http://m.bubuko.com/infodetail-3383948.html>

二、关闭线程池测试

通过分别设置如下的参数实现没有任务执行，核心线程打破等待循环

```

ksk.setKeepAliveTime(10, TimeUnit.SECONDS);
ksk.allowCoreThreadTimeOut(true);
  
```

因为使用了allowCoreThreadTimeOut方法，**KeepAliveTime必须大于0**，所以需要同时设置这两个参数。

同时为了满足flink job的需求，**KeepAliveTime**可以设为cp的时间间隔，这样可以保证旧的创建的线程池过期后自动关闭，且不影响正在运行的job，避免频繁创建线程的开销。

```

public void allowCoreThreadTimeOut(boolean value) {
    if (value && keepAliveTime <= 0)
        throw new IllegalArgumentException("Core threads must have nonzero keep alive times");
    if (value != allowCoreThreadTimeOut) {
        allowCoreThreadTimeOut = value;
        if (value)
            interruptIdleWorkers();
    }
}
  
```

```

1 class Person {
2     public ThreadPoolExecutor create(){
3         return new ThreadPoolExecutor(5,
4             5,
5             0L,
  
```

```

6         TimeUnit.MILLISECONDS,
7         new LinkedBlockingQueue<>());
8     }
9 }
10 public class test {
11     public static void main(String[] args) throws InterruptedException {
12         Person person = new Person();
13         ThreadPoolExecutor ksk = person.create();
14         ksk.setKeepAliveTime(10, TimeUnit.SECONDS);
15         ksk.allowCoreThreadTimeOut(true);
16         ksk.submit(() -> {
17             try {
18                 Thread.sleep(20000);
19                 System.out.println("运行结束");
20             } catch (InterruptedException e) {
21                 throw new RuntimeException(e);
22             }
23         });
24         Thread.sleep(40000);
25         System.gc();
26         Thread.sleep(300000);
27     }
28 }

```

可以发现，通过这样设置，线程池执行完后，等待10s后便会自己主动释放。

线程停止前

```

2022-08-30 16:52:35
Full thread dump OpenJDK 64-Bit Server VM (25.322-b06 mixed mode):

"Attach Listener" #14 daemon prio=9 os_prio=31 tid=0x00000001480fd800 nid=0xa703 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"pool-1-thread-2" #13 prio=5 os_prio=31 tid=0x000000013f191000 nid=0x5a03 waiting on condition [0x00000001700da000]
java.lang.Thread.State: TIMED_WAITING (parking)
  at sun.misc.Unsafe.park(Native Method)
  - parking to wait for <0x0000000795a249c8> (a java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject)
  at java.util.concurrent.locks.LockSupport.parkNanos(LockSupport.java:215)
  at java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject.awaitNanos(AbstractQueuedSynchronizer.java:2078)
  at java.util.concurrent.LinkedBlockingQueue.poll(LinkedBlockingQueue.java:467)
  at java.util.concurrent.ThreadPoolExecutor.getTask(ThreadPoolExecutor.java:1073)
  at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1134)
  at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
  at java.lang.Thread.run(Thread.java:750)

"pool-1-thread-1" #12 prio=5 os_prio=31 tid=0x000000013f190800 nid=0x5803 waiting on condition [0x000000016fece000]
java.lang.Thread.State: TIMED_WAITING (parking)
  at sun.misc.Unsafe.park(Native Method)
  - parking to wait for <0x0000000795a249c8> (a java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject)
  at java.util.concurrent.locks.LockSupport.parkNanos(LockSupport.java:215)
  at java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject.awaitNanos(AbstractQueuedSynchronizer.java:2078)
  at java.util.concurrent.LinkedBlockingQueue.poll(LinkedBlockingQueue.java:467)
  at java.util.concurrent.ThreadPoolExecutor.getTask(ThreadPoolExecutor.java:1073)
  at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1134)
  at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
  at java.lang.Thread.run(Thread.java:750)

"zhumingdada" #11 prio=5 os_prio=31 tid=0x000000012f0c1000 nid=0xa803 waiting on condition [0x000000016fcc2000]
java.lang.Thread.State: TIMED_WAITING (sleeping)
  at java.lang.Thread.sleep(Native Method)
  at com.zm.test$3.run(test.java:73)
  at java.lang.Thread.run(Thread.java:750)

"Service Thread" #10 daemon prio=9 os_prio=31 tid=0x000000013e823800 nid=0x3d03 runnable [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"C1 CompilerThread3" #9 daemon prio=9 os_prio=31 tid=0x000000012f05d800 nid=0x3e03 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

```

线程停止后

```

2022-08-30 16:53:17
Full thread dump OpenJDK 64-Bit Server VM (25.322-b06 mixed mode):

"Attach Listener" #14 daemon prio=9 os_prio=31 tid=0x00000001480fd800 nid=0xa703 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"Service Thread" #10 daemon prio=9 os_prio=31 tid=0x000000013e823800 nid=0x3d03 runnable [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"C1 CompilerThread3" #9 daemon prio=9 os_prio=31 tid=0x000000012f05d800 nid=0x3e03 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"C2 CompilerThread2" #8 daemon prio=9 os_prio=31 tid=0x000000014807b000 nid=0x3f03 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"C2 CompilerThread1" #7 daemon prio=9 os_prio=31 tid=0x000000014807a000 nid=0x4003 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

"C2 CompilerThread0" #6 daemon prio=9 os_prio=31 tid=0x0000000148079000 nid=0x4203 waiting on condition [0x0000000000000000]
java.lang.Thread.State: RUNNABLE

```

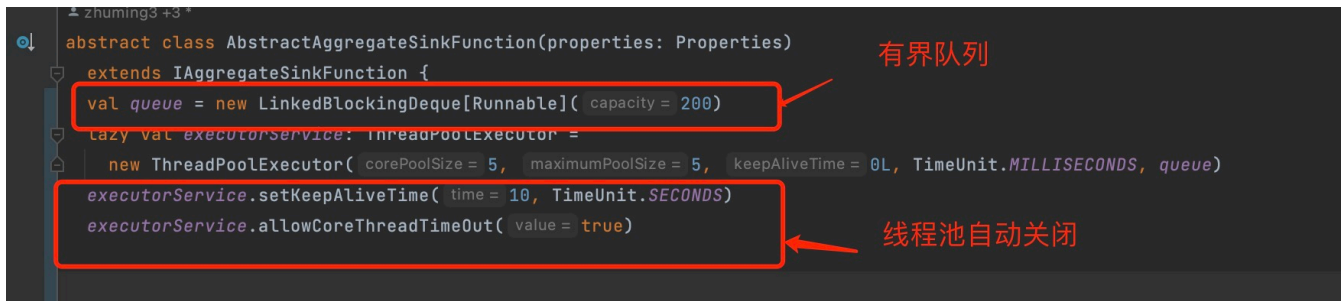
三、如何选择拒绝策略

在设置完有界队列和线程池自动关闭，线程池还有和拒绝策略相关，主要是保证在线程量激增的情况下，保证稳定，可以参考下面链接。

<https://www.cnblogs.com/javashare/p/15970569.html>

<http://m.bubuko.com/infodetail-3383948.html>

下面为我们已经设置完有界队列和线程池自动关闭



其中根据上面两个链接的介绍 CallerRunsPolicy 比较合适，下面主要验证下 CallerRunsPolicy 的模式的可性，从代码可以发现该模式下，当出现线程数大于（核心线程+队列大小）后会直接 **主进程执行job**。



下面为测试的线程池声明代码

```

ExecutorService es = new ThreadPoolExecutor(2, 2, 0L, TimeUnit.MILLISECONDS,
new ArrayBlockingQueue<Runnable>(3), Executors.defaultThreadFactory(),
new ThreadPoolExecutor.CallerRunsPolicy());

```

核心线程数位2，阻塞队列为3，拒绝策略为CallerRunsPolicy

其中t1和t2优先放入核心线程执行，t3、t4和t5放入queue中等待执行。

在t6放入时，触发拒绝策略，由主进程执行，可以看待执行结果id6的线程名为main，此时主进程变进入阻塞状态。

直达执行完后，print -----2，才进行t7和t8的执行，其中因为核心线程的存在，t7和t8先放入queue中等待前置t3、t4和t5执行完成。

这样的拒绝策略可以满足flink job的需求，即当queue中线程排满后，可以阻塞主进程来执行多出来的任务，即保证queue不无限膨胀，也可以防止超出queue后直接报错。只是为影响数据的lag量，最大限度保证系统可靠。

The screenshot shows the IntelliJ IDEA IDE with a project structure on the left, a code editor in the center, and a run console at the bottom. The code defines a `ThreadPoolExecutor` with `corePoolSize: 2`, `maximumPoolSize: 2`, and `keepAliveTime: 0L`. It uses `CallerRunsPolicy` for the `RejectedExecutionHandler`. Eight tasks (`t1` to `t8`) are submitted. A diagram on the right illustrates the state: a box labeled 'core' contains `t1` and `t2`, and a box labeled 'queue' contains `t3`, `t4`, and `t5`. Red arrows point from the `execute` calls for `t1` through `t5` to their respective boxes. A blue arrow points from the `execute` call for `t6` to the text 't6直接由主进程执行, 并阻塞主进程' (t6 is executed directly by the main process and blocks it). The run console shows the output of the tasks, including thread IDs and the main thread's ID.

```
public static void main(String[] args) {
    ExecutorService es = new ThreadPoolExecutor( corePoolSize: 2, maximumPoolSize: 2, keepAliveTime: 0L, TimeUnit.SECONDS,
        new ArrayBlockingQueue<Runnable>( capacity: 3), Executors.defaultThreadFactory(),
        new ThreadPoolExecutor.CallerRunsPolicy());

    MyTask t1 = new MyTask( id: "id:1");
    MyTask t2 = new MyTask( id: "id:2");
    MyTask t3 = new MyTask( id: "id:3");
    MyTask t4 = new MyTask( id: "id:4");
    MyTask t5 = new MyTask( id: "id:5");
    MyTask t6 = new MyTask( id: "id:6");
    MyTask t7 = new MyTask( id: "id:7");
    MyTask t8 = new MyTask( id: "id:8");

    es.execute(t1);
    es.execute(t2);
    es.execute(t3);
    es.execute(t4);
    es.execute(t5);

    System.out.println("-----1");
    es.execute(t6);
    System.out.println("-----2");
    es.execute(t7);
    es.execute(t8);
}
```

此时线程池

t6直接由主进程执行, 并阻塞主进程

Run: test x

/Users/zhuming/Library/Java/JavaVirtualMachines/liberica-1.8.0_322/bin/java ...

-----1

pool-1-thread-2, id: 2

pool-1-thread-1, id: 1

main, id: 6

-----2

pool-1-thread-1, id: 4

pool-1-thread-2, id: 3

pool-1-thread-1, id: 5

pool-1-thread-2, id: 7

pool-1-thread-1, id: 8

```
/Users/zhuming/Library/Java/JavaVirtualMac
-----1
pool-1-thread-2, id: 2
pool-1-thread-1, id: 1
main, id: 6
-----2
pool-1-thread-1, id: 4
pool-1-thread-2, id: 3
pool-1-thread-1, id: 5
pool-1-thread-2, id: 7
pool-1-thread-1, id: 8
```